

101 Belysning av klot

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, skärmhantering, simulering av en matematisk modell, utskrift på fil

Ett klot med centrum i origo och med radien r är belyst av ljus som faller in parallellt med den linje som går igenom punkten (x_0, y_0, z_0) på klotets yta och origo. Uppgiften är att skriva ett program som beräknar belysningsvariationen på klotet och som ger en snygg bild av det belysta klotet på datorn.

Belysningsintensiteten b i en punkt (x, y, z) på klotet är proportionell mot $\cos(v)$ där v är ljusets infallsvinkel i punkten. Låt proportionalitetskonstanten vara 1, vilket innebär att belysningen b kommer att anta värden mellan -1 och 1 .

Vid utskrift av belysningen används olika tecken för att markera olika ljusintensitet. Exempel på en teckenfördelning du kan använda dig av (den skall naturligtvis vara enkel att ändra i ditt program):

'M'	markerar mörker, dvs obelyst punkt där	$b \leq 0$
'*'	markerar en ganska mörk punkt	$0 < b \leq 0.3$
'+'	markerar en något ljusare punkt	$0.3 < b \leq 0.5$
'-'	markerar en ganska ljus punkt	$0.5 < b \leq 0.7$
'.'	markerar en ljus punkt	$0.7 < b \leq 0.9$
' '	markerar en mycket ljus punkt	$0.9 < b \leq 1$

Belysningen $b = \cos(v)$ i punkten (x, y, z) beräknas med skalärprodukten

$$b = (x \cdot x_0 + y \cdot y_0 + z \cdot z_0)/r^2$$

Skriv ett program som:

- Läser in värden på r (klotets radie), x_0, y_0 (definierar ljuskällan). I grunduppgiften kan dessa värden läsas in i terminalfönstret.
- Därefter beräknas z_0 enligt formeln.

$$z_0 = \sqrt{r^2 - x_0^2 - y_0^2}$$

- Därefter ska programmet genomlöpa alla x - och y -värden i intervallet $(-r, r)$ och för varje talpar (x, y) beräkna motsvarande positiva z -koordinat på klotytan. Tänk på att även här kontrollera rotuttrycket. Blir resultatet negativt under rotuttrycket, ligger punkten utanför klotets yta (låt $b = 0$ där).

$$z = \sqrt{r^2 - x^2 - y^2}$$

Dela upp intervallen i ett lagom stort antal steg (t.ex. 70). Det ska vara lätt att ändra antalet steg i både x - och y -led. Använd konstanter för detta.

- Lägg slutligen till möjligheten att välja att skriva ut klotbilden på fil.

Extrauppgift, betyg C: Kontrollerar indata, uttrycket nedan under rottecknet ska ej bli negativt. Om uttrycket under rottecknet blir negativt ligger ljuskällan felaktigt och nya indata för x_0 och y_0 måste inhämtas.

Extrauppgift, betyg B: Rita upp klotet med grafik. Använd t ex turtle eller tkinter. Du behöver inte lägga till grafiskt gränssnitt för inmatningen i denna del.

Extrauppgift, betyg A: Rita klotet i ett grafikfönster. Låt användaren flytta ljuskällan genom att klicka med musen på klotet. Ljuskällan ska bara flyttas om klickningen sker på klotet (inte om man pekar på bakgrunden), annars kommer det inte att gå att beräkna z_0 .

102 Museum

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, sökning, sortering, filhantering, grafik.

Skriv ett program för att hantera vad man kan se och uppleva för olika föremål på ett museum. Ett museum har vanligtvis ett antal utställningar med olika föremål, och som besökare är det självklart av stort intresse att kunna söka efter vilka föremål som museet ställer ut. Ett föremål har ett namn, unikt identifikationsnummer, beskrivning samt i vilken kontext det ställs ut (vikingatid, industriella revolutionen osv). Du skapar textfilen själv, och även om den inte behöver innehålla hundratals föremål, så ska programmet skrivas så att det kan hantera en stor mängd föremål.

Man ska kunna söka efter föremål utifrån namn och beskrivning och kontext (om flera föremål hittas skall alla presenteras för användaren), lägga till nya och ta bort föremål från museet, samt uppdatera föremåls beskrivning och kontext. Presentera de sökta föremålen på ett sätt som du finner lämpligt och överskådligt!

Vid sökning efter beskrivning, skall man kunna söka efter flera olika ord. Alla föremål som har en beskrivning som innehåller ett eller flera av de sökta orden skall presenteras i sorterad ordning, där föremålet med flest antal träffade sökord står överst.

Exempel på huvudmeny:

```
Museets hjälpreda! S Sök efter föremål... N Nytt föremål B Ta bort föremål
S Sluta.
```

Exempel på sökmenyn:

```
Museets föremålssökning! N Söka på föremålsnamn. S Sök föremål utifrån
sammanhang (kontextsök) B Fritextsök i föremålsbeskrivningar. T Tillbaka
till huvudmenyn...
```

Extrauppgift, betyg C: Inför felkontroll för användarens inmatning och filers existens.

Extrauppgift, betyg B: Låt programmet hålla reda på hur många gånger varje föremål sökts fram.

Ett föremål ska kunna ingå i flera samlingar (kontexter) samtidigt.

Det är vanligt att museer lånar ut föremål till varandra. Om ett museum lånat ut föremålet, så håller det så klart reda på vilket museum som lånat det. Detta betyder också att föremålet (när det dyker upp i sökning) måste visas med den extra informationen att det är utlånat. Man ska nu kunna välja att inte få med utlånade föremål i sökningen, att bara visa vilka föremål som är utlånade, samt som tidigare söka bland alla föremål.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till programmet. Användargränssnittet ska visa bilder på föremålen. Använd dig av bilder från Wikimedia Commons för att följa upphovsrättslagar.

105 Arga Troll

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, inläsning från tangentbord och filhantering.

Som vi alla vet så finns det många troll som bor i skogen och vissa av dessa troll kan vara ganska arga. Det är därför viktigt att trollen inte kan se några andra arga troll för då börjar de bråka. Detta är inte ett lika stort problem som man skulle kunna tro då troll är ganska dumma. De är väldigt bundna till naturen och tittar bara i de fyra väderstrecken samt diagonalerna mellan väderstrecken, sydväst, sydväst, osv.

Du ska göra ett spel där man ska placera ut arga troll på ett fyrkantigt bräde. Den som spelar spelet ska själv få välja hur stort brädet är (upp till en rimlig max-storlek).

```
- - - - -  
|_|_|_|*|_|  
|_|*|_|_|_|  
|_|_|_|_|_|  
|_|_|_|_|_|  
|_|_|_|_|_|  
|_|_|_|_|_|
```

Reglerna för spelet är ganska enkla. Det ska finnas:

- Ett troll per rad.
- Ett troll per kolumn.
- Inga troll får finnas på samma diagonal.

När man väl startat programmet så ska följande saker hända:

- Spelet börjar med att man hälsas välkommen till programmet och reglerna förklaras.
- Spelaren får välja storleken på bräde (minst 4x4).
- Spelaren får sedan börja placera ut trollen en taget, rad för rad. Spelaren ska kunna få ångra sitt val genom att skriva "undo" i stället för ett kolumnnummer (men om spelaren förlorat är det försent att ångra sig).

Du ska även ta tid för hur lång tid det tar för spelaren och lägga in det i en highscore-lista som sparas på fil. Placeringen ska både vara baserad på tid och storlek på brädet.

Extrauppgift, betyg C: Inför felhantering.

Extrauppgift, betyg B: Du ska nu skriva en algoritm som automatiskt löser problemet för små bräden, till exempel storlek 5 (kanske kan det klara av även 6 och 7). Algoritmen går ut på att man försöker att placera ut trollen rad för rad och om man misslyckas så backar man och testat nästa alternativ.

Extrauppgift, betyg A: Du ska göra ett GUI där spelaren får placera och tar bort troll från brädet genom att klicka på rutorna.

106 Damspel

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, inläsning från tangentbordet, filhantering.

Dam, damspel, är ett strategiskt brädspele som spelas av två spelare på ett kvadratisk bräde där användaren själv får välja storlek så länge det är en multipel av 2 och rimligt stor (en begränsning på 08-12 rekommenderas).

```

-----
|   | B |   | B |   | B |   | B |
-----
| B |   | B |   | B |   | B |   |
-----
|   | B |   | B |   | B |   | B |
-----
|   |   |   |   |   |   |   |   |
-----
|   |   |   |   |   |   |   |   |
-----
| V |   | V |   | V |   | V |   |
-----
|   | V |   | V |   | V |   | V |
-----
| V |   | V |   | V |   | V |   |
-----

```

Dam spelas av två personer. Det finns en rad olika regeluppsättningar och damvarianter. Gemensamt för dem alla är att man startar med sin halva av planen (minus raden närmast motståndaren) fylld med en pjäs på varannan ruta i form att ett schackmönster. Spelarna turas om med att göra drag varsitt drag. Spelaren ska få välja sin egen symbol. Målet är att fånga motståndarens brickor genom att hoppa över dem.

Reglerna för att göra ett drag är följande:

- Brickorna får bara hoppa diagonalt.
- Brickorna får bara hoppa en ruta per drag, om man inte tar en av motståndarens brickor.
- Man tar en bricka genom att hoppa över den. Brickor på kanten är därför säkra.
- Man måste ta motståndarens bricka om man kan och även fortsätta att ta så länge som det går.
- En bricka får bara hoppa framåt såvida den inte har nått hela vägen fram till motståndarens sista rad. Då får den hoppa baklänges också.

Den som mister alla sina brickor har förlorat.

Du ska även ta tid för hur lång tid det tar för spelaren och lägga in det i en highscore-lista som sparas på fil. Placeringen ska både vara baserad på tid och storlek på brädet.

Extrauppgift, betyg C: Inför felhantering.

Extrauppgift, betyg B: Du ska nu låta datorn spela. låt datorn ta så många pjäser som det går och om datorn inte kan ta något så slumpar den ett drag.

Extrauppgift, betyg A: Du ska nu göra ett grafiskt interface där spelaren gör sitt drag genom att klicka på pjäsen och sen klickar på vart han vill flytta den. Missa inte någon av den tidigare funktionaliteten i spelet.

111 När vi har tid

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, inläsning från tangentbord och filhantering.

När flera personer ska träffas samtidigt så är det inte alltid lätt att hitta den tid som passar bäst. Därför har du bestämt dig för att knåpa ihop ett finurligt lite program som löser det här problemet. Du ska skapa ett program som låter flera personer välja när de kan träffas.

Programmet består av fyra delar. Skapandet av träffar, angivelserna när man kan, summeringen och avslut.

Delarna ska se ut som följande:

- När man skapar en träff så får man ange två saker, namn på träffen och föreslagna tider (datum och tid) när man eventuellt ska träffas.
- När man ska ange när man kan så får man först ange namn och sen får man välja ett tillfälle som det hela gäller för. Efter det så får man tillfrågad om man kan vid alla angivna tillfällen men en ja/nej fråga.
- När man väljer summeringen och har valt träff så ska programmet räkna ut vilken tid som passar bäst och skriva ut alla som kunde träffas vid just det tillfället.
- När programmet avslutas skall all data sparas till fil så att vid nystart kan nya och gamla personer lägga till ny information till schemat.

Extrauppgift, betyg C: Inför felhantering.

Extrauppgift, betyg B: Du ska nu införa ja/nej/kanske istället för bara ja/nej som val. Du ska dessutom lägga till en ägare för varje träff och bara den personen ska kunna genomföra en summering (ingen säkerhet krävs i programmet).

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt för programmet.

112 Kontaktförmedling

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser. Filhantering. Datastrukturer. Sortering.

Du ska skriva ett program åt företaget Datakontakt. Företaget åtar sig att förmedla en lista med potentiella idealpartners. Vid en sådan förmedling får man först fylla i ett formulär av följande slag:

Vilket kön söker du? m (man) / k (kvinna) / i (icke-binär) / a (alla):

....

Inom vilket åldersspann söker du en partner (minst 18 år)?

Gradera från 0 till 5 hur du värdesätter följande:

Mognad

Skönhet

Intelligens

Humor

Förmögenhet

Utbildning

Du får själv skapa filen som ska användas, enligt följande format:

Format:

Namn(max 20 tkn),kön(m/k/i), ålder(år), mognad(0-1), skönhet(0-1), intelligens(runt 100),

humor(0-1),förmögenhet(kkr),utbildning(0-10)

=====

Hubert m 35 0.8 0.45 98 1.0 120 6

Erika k 24 0.7 0.85 101 1.0 3 2

Dana i 55 0.9 0.81 120 0.8 150 10

osv...

Programmet ska läsa in önskemålen (kön, ålder och betygen) enligt formuläret ovan. Programmet ska ta fram ett betyg på varje person genom att vikta varje egenskap med det tal som angivits i formuläret. De tio mest lämpliga personerna ska presenteras i betygsordning. I presentationen skall personernas data tas med. Vi använder oss av följande metod:

- Har personen rätt kön?
- Är personens ålder inom det angivna åldersspannet?
- Notera att värdena i textfilen är av olika storlek. För att räkna fram ett betyg behöver vi normera alla värden genom att dividera dem med respektive egenskaps maxvärde (t.ex. då intelligensmax bland de tre ovanstående personerna är 120, dividerar vi alla värden för intelligens med 120. Skönhetsmax är 0.85, dividera...). Detta betyder att de normerade värdena alltid kommer att vara tal mellan 0 och 1, samt vara relativa varandra.
- Därefter multiplicerar vi de normerade värdena med respektive vikt (värdet som användaren get kategorin) och summerar ihop dessa tal. Summan blir personens betyg.

Extrauppgift, betyg C: Inför fel- och rimlighetskontroll för alla inmatade värden.

Extrauppgift, betyg B: Se till att användaren också kan välja att lägga till eller ta bort en person ur registret. Filen ska givetvis uppdateras. Observera att det blir nödvändigt att normera om när personer läggs till eller tas bort.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt. Låt användaren mata in sina önskemål genom att fylla i ett formulär med textfält, knappar etc. Kanske vill du lägga in foton?

De delar du gjorde i B-uppgiften måste inte tas med i programmet för A-delen.

120 Parkeringshuset

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, datastrukturer, sortering.

Ett parkeringshus har bestämt sig för att bygga ett nytt system för sina trognaste kunder för att underlätta betalningarna och förhoppningsvis få dem att parkera där oftare. Parkeringshuset vill också vara miljövänliga så man har bestämt sig för att ta betalt beroende på hur stor bil folk har.

I kundsystemet så har man följande information som sparas på fil mellan programkörningarna: Registreringsnummer, biltyp (liten, mellan eller stor) samt ägaren av bilen. Ett prisexempel kan vara att små bilar kostar 20kr/h, mellanstora 25kr/h och stora kostar 30kr/h. Ditt program ska också hålla koll på in- och utpassager för varje bil och läsa in all historik för detta när programmet startas (kan finnas i samma fil eller en egen fil).

Vid programkörning ska man från huvudmenyn kunna välja att mata in information om när bilen kom till och lämnade parkeringshuset. Då ska först datum anges och sedan klockslag för både in- och utpassage (endast parkering under samma datum är tillåten). När programmet avslutas ska alla inlästa och inmatade in- och utpassager (datum och klockslag) skrivas ner på fil, så att de kan läsas in nästa gång programmet körs.

Kunder ska ges möjlighet att få veta hur stor deras skuld för en bil till parkeringshuset är. Parkeringshuset har valt att avrunda upp alla parkeringstider till närmast halvtimme. Om kunden ifrågasätter priset, så ska även historiken för en bil kunna visas!

Exempel på inmatning av parkerande bil:

```
Välkommen till P-(uppgifts)-huset!
I  Inpassage/Utpassage...
N  Lägg till ny bil
P  Räkna ut parkeringskostnad för en bil
V  Visa parkeringshistorik för en bil
S  Avsluta.
>>> I

Inpassage/Utpassage!
Ange registreringsnummer:
>>> AAA123
Ange datum:
>>> 2024-06-25
Ange starttid:
>>> 00:42
Ange sluttid:
>>> 13:37
OK!

---> ... tillbaka till huvudmenyn
```

Extrauppgift, betyg C: Inför felhantering av all inmatning till programmet, inklusive att datum och tid som anges är rimliga. Kontrollera också filen/filer som ska läsas från verkligen finns.

Extrauppgift, betyg B: Utöka menyn med alternativet att låta kunden betala sin skuld. Skulden ska beräknas på de in- och utpasseringar som bilen gjort. Om kunden betalar mer, så är det pengar till godo som ska kunna användas vid framtida parkeringar. Eftersom parkeringsbolaget ingått hemligt avtal med dataövervakningsmyndigheten får ingen data raderas ur historiken som finns lagrad på fil.

Lägg också till så att det nu går att parkera även över datumgränserna, t.ex. komma till parkeringshuset 12:34 2024-06-24 och åka 2025-06-25 17:01. För att detta ska fungera behöver användaren få mata in datum för både in- och utpassage, dessa båda datum ska också sparas i historiken.

Extrauppgift, betyg A: Lägg till ett grafiskt gränssnitt.

122 Livet hos matematiska organismer

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, filhantering.

En familj organismer består av celler (rutor) i ett rutnät som kan beskrivas av en matris. Cellerna föds och dör enligt bestämda levnadsregler och vi vill undersöka hur familjens fortbestånd förändras under ett antal generationer. Varje cell har åtta grannar:

```
1 2 3
4 * 5
6 7 8
```

Överlevnad

- Varje cell med två eller tre levande grannar kommer att överleva till nästa generation.
- En cell med mer än tre grannar dör av överbefolkning.
- En cell med mindre än två grannar dör av ensamhet.

Födelse

- En tom cell som har exakt tre grannar kommer att födas och bli en levande cell i nästa generation.

Skriv ett program som först läser in antalet önskade generationer samt storleken på matrisen. För att slippa tråkiga och tidsödande inmatningar ska du lägga upp filer med cellernas koordinater. En fil kan se ut så här:

```
Format: x-koord y-koord
=====
2 3
2 4
2 5
⋮
```

Presentationen kan enklast ske via ascii-grafik, dvs vanliga tecken och blanka skrivs ut så att de tillsammans ser ut som en enkel figur.

Programmet skall vid en sådan presentation rita ut matrisen och dess levande innevånare på ett illustrativt sätt. Det är lämpligt att lägga in en pausfunktion så att användaren hinner se alla utritningar, t.ex. att användaren trycker retur innan nästa bild ritas upp.

Tips: Antag att du har valt att använda dig av en 15x15-matris. För att undvika vissa kons- tigheter ute i kanterna kan du då jobba med en 17x17-matris, detta för att kunna införa fiktiva celler i 0:te och 16:e position (så att alla verkliga celler får 8 grannar).

Exempel på starttillstånd som ger en trevlig fortsättning:

18 levande celler:

```
-----  
-----  
---*---*---  
---****---  
--*-----*--  
--*-***-*--  
--*-----*--  
---****---  
-----  
-----
```

CHESHIREKATT. Utskrift varje generation.
Lägg katten mitt i rutnätet. Efter några
generationer är bara leendet kvar.

5 levande celler:

```
-*-----  
--*-----  
***-----  
-----  
-----
```

GLIDARE.

Vid utskrift av var 4:e generation
glider denna figur snett nedåt.

9 levande celler:

```
----*-----  
-----*-----  
*-----*-----  
-*****-----  
-----
```

RYMDSKEPP.

Flyttar sig horisontellt om
utskrift görs var 4:e generation.

Extrauppgift, betyg C: Programmet ska kontrollera att koordinaterna ligger inom tillåtet intervall. Användaren skall även kunna bestämma hur ofta han/hon vill se processens fortskridande. Användaren matar in ett heltal, ex. 4 , vilket tolkas som att användaren vill se var 4:e generation.

Extrauppgift, betyg B: Inför följande förbättringar av ditt program

- I stora tomma områden är det onödigt att kontrollera varje cell. Hitta på ett sätt att hålla reda på vilka delar som är just nu inaktiva, och hoppa dessa vid genomgången.
- Istället för att ha döda kanter kan man tänka sig att matrisen är klistrad på en torus så att vänster kant sitter ihop med höger (och övre kanten med den undre). Inför detta och hitta ett starttillstånd som demonstrerar övergången.

Extrauppgift, betyg A: Gör ett grafiskt gränssnitt, där förloppet ritas ut i ett rutnät istället. Lägg också till två knappar och ett textfält; En av knapparna ska stega bilden till nästan utritning, den andra ska rensa matrisen och i textfältet ska användaren mata in hur många generationer som ska gå mellan utritningarna. Gör också så att man kan skapa och/eller döda celler genom att trycka på rutorna.

Datafiler och hjälpfiler: cheshirekatt.txt

glidare.txt

rymdskepp.txt

124 Packlistor för resor

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, datastrukturer, sortering.

Vid olika resor, utflykter och andra tillfällen när man behöver komma ihåg ett större antal föremål än hjärncellerna räcker till för vore det trevligt om det fanns ett program som kunde vara till hjälp för att komma ihåg alla prylar.

Vid körning av programmet ska användaren kunna skapa nya packlistor, och sedan hålla reda på vilka saker som ska packas med för respektive resa genom att lägga till påminnelser till respektive packlista. Man ska även kunna ta bort föremål som skrevs upp felaktigt.

Användaren ska kunna visa alla saker som står på en viss lista genom att söka efter hela (eller delar) av namnet på packlistan. Om flera packlistor hittas skall användaren med ett menyval få välja en dessa, som sedan skrivs ut.

För att veta vilka aktuella packlistor som finns skall programmet kunna skriva ut samtliga listors namn och datum, sorterat efter just datum. Användaren anger ett datum, och alla packlistor som har ett datum lika med eller efter detta skall skrivas ut. Lämnar man datumet blankt visas alla listor i systemet.

När programmet avslutas ska all information skrivas till en textfil som läses in när programmet startar. Ett förslag på format i textfilen kan vara:

```
Spelning Gamla Stan;20150902
Trummor;Konfetti;Två burkar hallonsaft;Karta över Australien;Konstigt instrument
Maratonlöpning i historiska fotspår;20150101
Kappa;Skor;Hög hatt;Fickur;Promenadkäpp
Utflykt till gamla Trollskogen;20150722
Yllefilt;Engångsgrill;Kol;En liten elfläkt(batteridriven);Kakor;Osynlighetsmantel
```

Förslag på huvudmeny och visning av en lista:

```
Välkommen till Packningsprogrammet!
```

```
N Ny packlista
I Visa innehåll i lista
S Lägg till sak till en lista
A Visa alla kommande listor
S Avsluta.
>>> I
```

```
Visa lista!
Vilket namn på lista letar du efter?
>>> gamla
```

```
Det finns flera listor som matchar din sökning:
```

```
1. Spelning Gamla Stan - 20150902
2. Utflykt till gamla Trollskogen - 20150722
>>> 2
```

```
Dessa saker ska packas med!  
* Yllefilt  
* Engångsgrill  
* Kol  
* En liten men effektiv elfläkt(batteridriven)  
* Kakor  
* Osynlighetsmantel  
  
... ---> tillbaka till huvudmenyn
```

Förslag på huvudmeny och att skapa en ny packlista:

```
Välkommen till Packningsprogrammet!  
N Ny packlista  
I Visa innehåll i lista  
S Lägg till sak till en lista  
A Visa alla kommande listor  
S Avsluta.  
>>> N  
  
Skapa ny lista!  
Vad ska listan heta?  
>>> Repetition med bandet  
  
Vilket datum gäller listan?  
>>> 20151011  
  
Listan skapades!  
  
... ---> tillbaka till huvudmenyn
```

Tips: Låt användaren mata in dagens datum vid start av programmet.

Extrauppgift, betyg C: Inför felhantering i programmet, som kollar att filer man angav verkligen finns, och att formaten på inmatade strängar stämmer. Se även till att datum (välj själv på vilken form och visa detta klart och tydligt för användaren) matas in korrekt, och att datumen faktiskt existerar. Om fel hittas i textfilen ska programmet upplysa användaren om att fel existerar, på vilken rad felet är, samt hoppa över hela den packlistan och köra vidare.

Extrauppgift, betyg B: Utöka funktionaliteten till att man nu ska kunna markera föremål som packade. Ge användaren möjlighet att se antingen alla föremål i en specifik packlista, eller bara de föremål som inte prickats av och packats med ännu.

Alla föremål ska presenteras i bokstavsordning!

Vid sökning efter listor ska även datumet tas i beaktande. Om flera packlistor gäller samma dag, skall på samma sätt som tidigare en väljas ut och presenteras.

Extrauppgift, betyg A: Lägg till ett grafiskt gränssnitt.

126 Dalek

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, objekt

Doktorn är jagad av en grupp Daleks i en labyrinth. Så här ska spelet fungera:

- För varje steg som Doktorn tar, tar alla Daleks ett steg mot Doktorn.
- Om två Daleks krockar, så ligger det kvar en hög med skrot. Doktorn kan inte gå på högen med skrot – Daleks som åker på den försvinner ur spel.
- Doktorn kan teleportera sig med sin Sonic Screwdriver till en slumpmässig ruta.
- Krockar någon Dalek med Doktorn så förlorar spelaren.
- Har alla Dalek kraschat så vinner spelaren.

Spelet utspelar sig på ett rutnät. Vissa rutor är väggar – alla andra rutor är golv. Doktorn finns på en ruta, varje Dalek på en ruta.

Doktorn flyttar sig enligt styr-kommandon från spelaren. Daleks går så rakt mot Doktorn de kan – genom att ta ett steg till någon av de närmaste 8 rutorna.

Om närmaste riktningen går in i en vägg, så kommer Daleken stå stilla (om den ska rakt in i väggen) eller följa väggen (om den ska snett in i väggen).

Labyrinten finns i en fil: filnamnet matas in av användaren för att starta spelet. Filen innehåller en labyrinth beskriven som lika långa rader med . för golv, * för vägg, A för Dalek, # för skrothög och D för Doktorn. Exempelvis kan det se ut så här:

```
*****
*. . . . .*.
*. . . . .*.*****.
*. . . A. . . . .*. . . . . A. . .
*. . . . . D. . . . .*. . . . .
*. . . . *****.*****.
*. . . . .*. . . . .*.
*. . . . .*. . . . A. . . . . A. . .
*. . . . *****.
*. . . . .*.
*****
```

Extrauppgift, betyg C: Hantera fel i inmatning och i labyrinthfilen: om användaren matar in ett fel så ska programmet beskriva felet och be om ny inmatning.

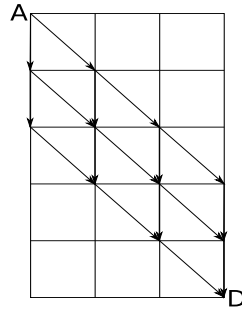
Labyrinthfilen är korrekt om:

- Alla rader är lika långa
- Alla tecken är något av . * # D A

Om labyrinthfilen inte är korrekt ska programmet beskriva felet och be om en ny labyrinthfil.

Extrauppgift, betyg B: Om riktningen är uppenbar, dvs $\text{abs}(\text{dx})$ och $\text{abs}(\text{dy})$ är antingen lika med varandra eller 0, så ska programmet göra som i uppgiftens E-del. Annars ska programmet slumpa fram en riktning.

Doktorn är 3 steg till höger och 5 steg ner för Daleken. För att välja steg så slår Daleken fram ett slumpstal r . Om $r < 3/5$ så går Daleken snett åt höger, om $r > 3/5$, rakt ner.



Extrauppgift, betyg A: Skapa ett grafiskt gränssnitt, exempelvis med någon av Python-modulerna `curses`, `pygame` eller `tkinter`.

127 Platsbokning på SJ

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, datastrukturer.

Skriv ett program som hjälper SJ med platsbokning i en tågvagn med n antal platser (3 i exemplet nedan). På skärmen ska en bild, liknande den nedanstående, ritas upp med det aktuella bokningsläget. Observera den fiffiga numreringen som gör att platsnummer i följd ger intilliggande platser. Modulo kan vara till hjälp för att få till kolumner och rader. Välj själv om vagnen skrivs ut antingen på bredden eller på höjden. Redan bokade platsnummer markeras på lämpligt sätt på skärmen, t ex genom att numret omges med två stycken '*'.

1	8	9	16	17	24	25	32
2	7	10	15	18	23	26	31
TYST AVD							
3	6	11	14	19	22	27	30
4	5	12	13	20	21	28	29

Med hjälp av följande meny ska användaren kunna boka respektive avboka platser samt skriva ut biljetter för de senaste bokningarna (de som bokats under denna körning). Tänk på att redan bokade platser inte ska kunna bokas dubbelbokas, så du behöver läsa in vilka platser som är bokade från fil vid programstart. Du ska heller inte kunna boka fler platser än det finns lediga.

Vad vill du göra?

- Boka, skriv 'B', på samma rad följt av önskat antal biljetter.
- Avboka, skriv 'A', på samma rad följt av ett platsnummer.
- Skriva ut de senast bokade biljetterna, skriv 'S'.
- Avsluta, skriv 'Q'

Ditt val:

Efter varje bokning / avbokning ska bokningsläget uppdateras på skärmen och innan programmet avslutats även sparas på fil inför nästa programkörning.

Detta program behöver bara skriva ut biljetter på en enda sträcka, t. ex. mellan Stockholm och Göteborg. Platsbiljetterna skrivs ut på en egen "biljettfil" med exempelvis följande utseende:

```
PLATSBILJETT
Sth-Gbg 9.05
Plats 19
TYST AVDELNING
Mittgång
```

VGv

Extrauppgift, betyg C: Inför felkontroll för all inmatning till programmet.

Extrauppgift, betyg B: Inför fler sträckor och avgångstider. Ett tåg består rimligtvis av flera vagnar som kan ha olika antal platser, så uppdatera ditt program så att det nu bokar platser på tåg. Ett tåg har ett unikt tågnummer, som kan användas för att hålla ordning på textfilerna.

Försök se till att bokade platser hamnar intill varann. Annars ska programmet fråga om det är OK med spridda platser.

Bokningsläget (som består av de bokade platsernas nummer) för varje sträcka, tåg och avgång ska sparas på fil.

När tid och datum för avgången passerat tas filen bort.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt. Alla bokningar ska ske via musklickningar. Lägg också till en knapp för att skriva ut bokade biljetter.

128 Prickskytte

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, filhantering, sortering.

En femringad måltavla har cirklar med radierna 10, 30, 50, 70 och 90 mm. Träff innanför den innersta cirkeln ger 5 poäng, övriga träffar i radiell ordning från centrum ger 4, 3, 2 respektive 1 poäng. Träffar utanför den yttersta ringen ger förstås 0 poäng. Vi kan även använda en formel för beräkningen av antal poäng p :

$$\begin{aligned} p &= (110 - \text{radien}) \text{ DIV } 20 & 0 \leq \text{radien} \leq 90 \\ p &= 0 & \text{radien} > 90 \end{aligned} \quad \text{där DIV betyder heltalsdivision.}$$

Din uppgift är att simulera en tävling i prickskytte. En skottserie består av tio skott och vid beräkning av poängsumman räknar man bort de två sämsta skotten. Får de två (eller fler) bästa skyttarna samma poängsumma så blir det omskjutning mellan dessa.

Koordinaterna (x, y) för träffarna slumpas fram med hjälp av ett normalfördelat slumpstal (se `random.txt`). I pythons `random`-modul finns funktionen

```
random.normalvariate(mu, sigma)
```

som ger ett normalfördelat slumpstal. Parametrarna `mu` och `sigma` motsvarar medelvärdet respektive standardavvikelsen. (dvs hur mycket kasten avviker från medelvärdet). (Om du vill veta mer om medelvärde, standardavvikelse och normalfördelning så kan du läsa på i Wikipedia.)

Skriv ett program som läser in ett godtyckligt antal deltagare från en textfil, som du själv skriver ihop. Denna kan se ut så här:

Format:

```
Namn (max 20 tkn) ; sigma ; antal genomförda tävlingar  
antal vunna tävlingar ; medelresultat på tävlingarna
```

```
=====
```

```
Pelle Pajas;32;12;2;24.3
```

```
Kalle Exakt;1;12;11;35.7
```

```
⋮
```

Användaren anger hur många tävlingar som ska genomföras under säsongen och därefter tävlar de tävlande mot varandra i tävling efter tävling tills säsongen är över. Vid varje tävling ska deltagarnas 10 skott skrivas ut följt av deras resultat. Efter avslutad tävling ska vinnaren skrivas ut. När säsongen är avslutad, ska de tävlandes statistik skrivas ut (antal vinster). Det kan se ut så här:

```
=====
Tävling nr 1:
Kalle Exakt  4  5  5  5  4  5  5  5  4  4  resultat  = 38
Pelle Pajas  1  3  4  2  2  3  5  2  3  4  resultat  = 26
Nisse Hurra  3  3  5  1  5  5  5  5  5  5  resultat  = 38
:
Kalle Exakt och Nisse Hurra hade båda 38 poäng (bäst i tävlingen).
De skjuter om:
Kalle Exakt  4  5  5  4  5  5  5  4  4  4  resultat  = 37
Nisse Hurra  5  5  5  1  5  5  5  5  1  5  resultat  = 40
Nisse Hurra vann tävling nr 1.
=====
Tävling nr 2:
Kalle Exakt  4  3  5  4  5  5  3  3  5  5  resultat  = 36
:
=====
Statistik:
Kalle Exakt  vann 5 tävlingar.
Pelle Pajas  vann 1 tävling.
:
```

Lägg in en pausfunktion efter ett lämpligt antal tävlingar (annars hinner vi ju inte se alla resultat).

Hantering av delad seger samt uppdatering av filen ingår inte i grunduppgiften.

Extrauppgift, betyg C: Inför möjligheten att lägga till nya tävlande med hjälp av input. Inför även fel och rimlighetskontroll av värdena som användaren skriver in.

Extrauppgift, betyg B: Se till att programmet kan hantera delad seger (se exemplet ovan).

Uppdatera filen efter avslutad säsong med de nya värdena på antal genomförda tävlingar, antal vunna tävlingar och medelresultat för varje tävlande.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt där användaren kan välja hur resultaten ska visas.

Datafiler och hjälpfiler: random.txt

133 Sänka skepp

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer. Viss skärmhantering. I extrauppgiften tillkommer sortering.

Du ska skriva ett program som lägger ut en mängd fartyg på en yta (fartygspositioner läses in från fil). Användaren ska sedan kunna beskjuta ytan och programmet ska meddela resultatet (envägs beskjutning). Spelet avslutas då användaren så önskar eller efter att alla fartyg är sänkta.

Programmet skall efter varje skott:

- Skriva ut resultatet av skottet, träff eller bom.

Användaren ska via menyer kunna göra sin olika val. Huvudmenyn kan se ut så här:

Dina valmöjligheter (1-3):

- 1) Beskjuta fiendefartyg
- 2) Fuska lite, tjuvkika på fiendefartygen
- 3) Avsluta

Ditt val (1-3):

De olika menyerna innebär:

Beskjutning mot fiendefartygen: På skärmen ska spelplanen och skottmarkeringarna visas i en schackbrädesliknande figur. Användaren ska få skjuta ett eller flera skott och därefter återgå till huvudmenyn. Under spelplanen ska träffprocenten skrivas ut.

Fuska lite, tjuvkika på fiendefartygen: Alla spelare har olika moral och alla måste tillfredsställas. Spelplanen presenteras med fiendefartygen fullt tydligt markerade tillsammans med skottmarkeringarna.

Avsluta: Användaren ska få möjlighet att titta på de ej sänkta fartygens lägen före avslutning. Spelplanen presenteras på samma sätt som vid tjuvkiken.

Du bestämmer, som programkonstruktör, enväldigt spelplanens storlek (t ex 8x8). Fartygens positioner läses in från fil.

För att göra spelet mindre förutsägbart ska du ha minst tio olika filer med fartygspositioneringar. Programmet ska slumpa fram vilken fil som ska användas. I filen `random.txt` får du tips till slumpfunktionen.

I presentationen av spelplanen ska det tydligt framgå:

- Rutnätet, varje koordinat har en egen ruta, se bilden nedan.
- Resultaten av skotten, t.ex.

Träffar markeras med '#'

Bommar med 'o'

Fartyg , (ej träffade) med 'X'

- Koordinater i kanterna.

		1	2	3	4	

	A	O	X	O		

	B		#		X	

	C		O		X	

Extrauppgift, betyg C: Inför felkontroll av menyalternativ och inmatning.

Skriva ut ett felmeddelande om skottet var ogiltigt, dvs utanför planen eller rutan redan beskjuten. Användaren får i detta fall skjuta på nytt.

Extrauppgift, betyg B: Slumpa fram fartygens positioner istället för att läsa från fil. Du bestämmer fartygens antal samt storlek, vilket presenteras i början av programmet.

Du skall programmera på ett sådant sätt att man i efterhand lätt kan förändra fartygsuppsättningen.

Regler för placering av fartygen:

- Det ska finnas fartyg av olika längder, 1 - 5 rutor
- Fartygen ska kunna hamna både lodrätt och vågrätt
- Fartygen får inte överlappa
- Två fartyg får aldrig ligga precis intill varandra, utan det måste alltid finnas tomma rutor emellan.

Om det blev en träff och fartygets samtliga rutor är träffade är fartyget sänkt. Detta ska meddelas användaren. Dessutom ska programmet markera "beskjuten" på alla kringliggande rutor eftersom det inte kan ligga några fartyg där.

Programmet ska dessutom uppdatera en high-score mellan körningarna. Om spelaren är bland de tio bästa (träffprocenten är måttet på spelarens skicklighet) ska namn efterfrågas. Listan lagras efter körningen på en textfil. Denna läses in vid ny programstart. Listan ska sorteras.

Extrauppgift, betyg A: Gör ett grafiskt gränssnitt där man klickar för att beskjuta.

Datafiler och hjälpfiler: random.txt

134 Springarens vandring på schackbrädet

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, randvillkor.

Skriv ett program som följer en springares vandring över ett 64 rutors schackbräde! Springaren följer schackreglerna, dvs den förflyttar sig antingen två steg horisontellt och ett steg vertikalt eller två steg vertikalt och ett steg horisontellt. Under hela vandringen får springaren inte komma till en plats mer än en gång.

Användaren får ange vilken ruta springaren ska starta på. Det finns högst 8 stycken platser som springaren kan gå till från varje ruta. Du skriver lämpligen en klass vars fält representerar schackbrädet och vars metoder tex. förflyttar springaren. Står vi på en kantruta kan vi inte flytta springaren i vissa riktningar, ty då hamnar vi utanför brädet. I tipset finner du en variant att lösa detta problem. Programmet ska plocka ut de godkända alternativen och sedan med hjälp av slumpen välja vilken ruta som springaren ska gå till.

Exempel:

```
nyPlats = random.randrange(antal0Kplatser)
```

Du får numrera de godkända platserna enligt en fast ordning, tex medsols. Programmet tar slut då springaren inte har någon plats att flytta till. Användaren ska starta programmet genom att ange en startpunkt på schackbrädet. Därefter ska brädet skrivas ut. I presentationen av schackbrädet ska det tydligt framgå:

- Rutnätet, varje koordinat har en egen ruta.
- Springarens förflyttningar ska vara markerade, en ruta kan då innehålla:

Tomrum , springaren har inte besökt denna ruta.

Ett tal , springaren har besökt denna ruta, i steg nr ...

- Koordinater i kanterna.

Tips: Schackbrädet kan lämpligen representeras av en matris som förutom det verkliga schackbrädet (rad 2 – 9, kolumn 2 – 9) har ytterligare två rader och kolumner runt omkring (som flyttbuffert). Matrisen får då indexgränserna (0..11). På raderna 0 och 1 såväl som i kolumnerna 10 och 11 sätts alla element till -1 för att markera att rutorna ligger utanför brädet. För att markera rutornas tillstånd ska du använda dig av någon konvention.

T ex kan olika tal ge information om rutan:

-1 Rutan ligger utanför schackbrädet.

0 Rutan ligger på schackbrädet, springaren har aldrig varit här.

k Rutan ligger på schackbrädet, springaren har varit här tidigare, i steg nummer k (där k är ett tal > 0).

Utskriften från programmet kan se ut så här:

Springarens vandring:

8					2												

7							3										

6				1													

5																	

4																	

3																	

2																	

1																	

	A	B	C	D	E	F	G	H									

Extrauppgift, betyg C: Ge användaren möjlighet att mata in en egen springarvandring. Kontrollera att inmatade koordinater motsvarar en tillåten ruta.

Extrauppgift, betyg B: Lägg till ett alternativ där användaren kan välja att få se statistik. Simulera 1000 springarvandringar och samla ihop statistik över hur många rutor springaren passerade i varje simulering.

Extrauppgift, betyg A: Ge programmet en snyggare presentation genom att visa resultatet i ett separat grafikfönster. Låt användaren få se ett steg i taget.

137 Solkraftverk

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, sortering, datastrukturer.

Man ämnar bygga ett solkraftverk och vill därför simulera kraftverkets funktion under ett år. Den energi som utvinns ur kraftverket beror på en del olika faktorer. Vi ställer upp följande modell för utvunnen energi under dagen t :

$$W(t) = area \cdot soltal \cdot solighetsfaktor \cdot f(t, latitud)$$

Area Solfångarens area (några 100-tal kvadratmeter)

Soltal Proportionalitetskonstant, energi/ytenhet (några kWh/kvadratmeter)

Solighetsfaktor Slumpas fram som ett tal mellan 0 och 1 (se filen `random.txt`). Slumpa en ny solighetsfaktor för varje dag och latitud.

$f(t, latitud)$ Energifunktion. Latituden anges av användaren, tiden t varierar mellan 1 och 360 (dagarna under ett ett år)

Låt:

$$v = \frac{(23.5 \cdot \sin(\frac{\pi \cdot (t-80)}{180}) + 90 - latitud)}{90} : 0 < latitud < 90$$

Då gäller:

$$\begin{aligned} f(t, latitud) &= v^2 & , & \quad 0 < v < 1 \\ f(t, latitud) &= 1 & , & \quad v \geq 1 \\ f(t, latitud) &= 0 & , & \quad v \leq 0 \end{aligned}$$

Man kan anta förenklingen att alla månader innehåller 30 dagar och året 360. Bolaget som ska bygga kraftverket vill med hjälp av denna modell ta fram en optimal placering av kraftverket (den placering som ger mest energi). Användaren ska därför kunna mata in olika latituder och få ut energivärden. Programmet ska kunna beräkna utvunnen energi för ett godtyckligt antal latituder. Slutresultatet presenteras, sorterat, i tabellform med latitudernas årsmedelproduktioner (summan av dagsproduktionerna/360).

Tips: Medelvärde (x_m) och standardavvikelsen (s) beräknas med följande formler:

$$x_m = \frac{1}{n} \sum_{i=1}^n x_i$$

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - x_m)^2}$$

Extrauppgift, betyg C: Programmet ska skapa en fil som kan se ut så här:

Format:

Area,soltal,latitud,dag,solighetsfaktor,f(t,latitud),W(t)

=====

Januari:

450 7 56 1 0.5 0.1215 23.25

450 7 56 2 0.2 0.1224 9.44

450 7 56 3 0.9 0.1234 43.17

⋮

Här presenteras varje dag som en rad. För att underlätta för den som ska läsa av tabellen ska månadernas namn skrivas ut.

Se också till att felhantera all indata som du får till programmet.

Extrauppgift, betyg B: Inför vindkraftverk som alternativ. Energin från vindkraftverket varierar med tiden (det blåser mer under vår och höst) och rotordiameter (ju större rotor desto högre effekt). Rotordiametern ligger mellan 25 och 50 meter och effekten ligger kring 500 kW.

Extrauppgift, betyg A: Skapa ett grafiskt användargränssnitt där användaren ska kunna detaljgranska en latituds värden (under ett år) antingen grafiskt (stapeldiagram) eller siffermässigt (tabell), bägge valmöjligheterna måste finnas med. Minsta tidsenhet vid presentationen är månader; hela året ska granskas. Väljer användaren att få presentationen på tabellform ska varje presenterat medelvärde åtföljas av standardavvikelsen, samt min- och maxvärdet för aktuell period.

Datafiler och hjälpfiler: random.txt

139 Tidtabeller för en pendeltågslinje

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, listor.

På en pendeltågslinje sätter man då och då in nya lok med allt starkare motorer. Det betyder att tidtabellen måste förnyas varje gång. Din uppgift blir att skriva ett program som tillverkar nya tidtabeller för linjen. Tidtabellen ska innehålla stationsnamn och avgångstider för varje station längs linjen, t ex Floda 09.54.

Indata är stationsnamnen, avstånden från startstationen till de olika stationerna på linjen samt data på tågets prestanda. För att få med av- och påstigningstiden ska du använda följande modell, som ger väntetider på 0–1 minuter:

$$\text{avgångstiden} = \text{heltalsdelen av ankomsttiden} + 1 \text{ (minuter)}$$

Lägg upp en fil med alla indatavärden och ändra i denna då tågprestanda eller linjesträckningen ska förnyas. Exempel på hur filen kan se ut:

Prestanda, format:

acceleration (0.4 - 0.8 m/s²); retardation (1.2 - 2.0 m/s²);
maxhastighet (30 - 45 m/s)

=====

0.5;1.5;35

=====

Tabellformat:

antal stationer

stationsnamn (max 15 tkn) / avstånd från startstationen (km):

=====

11

Arby 0

Bedinge 3.57

Cebro 6.38

Degum 8.67

Ekö 12.80

Floda 18.24

Guldö 24.59

Håsta 29.42

Iby 30.73

Jituna 34.06

Kåvik 36.95

Alla dagar

Endast ett tåg är i trafik. Första avgången mot Kåvik är kl 08.35 från Arby. När tåget anlänt till Kåvik, väntar det på stationen i 10 minuter innan det vänder tillbaka mot Arby. Vid ändstationerna beräknas alltid avgångstiden ligga 10 minuter efter ankomsttiden. Sista tåget från Kåvik ska ha en avgångstid före midnatt. Efter ankomst till Arby står det kvar där över natten. Detta innebär att tåget kommer att åka fram och tillbaka ganska många gånger.

Naturligtvis ska det vara någorlunda lätt att ändra dessa data. Man skulle kunna starta tågen senare, låta tåget vänta längre på station osv.

Tågets prestanda anges med följande parametrar:

a	tågets acceleration (m/s^2)
r	tågets retardation (m/s^2)
v_{max}	tågets maxhastighet (m/s)
s	sträckan mellan två stationer (m)
s_0	sträckan som tåget behöver för att nå sin maxhastighet $\frac{v_{max}^2}{2 \cdot a}$ (m)
s_1	sträckan som tåget behöver för att bromsa till stillastående vid maxhastighet $\frac{v_{max}^2}{2 \cdot r}$ (m)

För restiderna mellan två stationer gäller då:

$$t = \begin{cases} \sqrt{2 \cdot s \left(\frac{1}{a} + \frac{1}{r} \right)} & \text{för } s \leq s_0. \\ v_{max} \cdot \left(\frac{1}{a} + \frac{1}{r} \right) + \frac{s - s_0 - s_1}{v_{max}} & \text{för } s > s_0. \end{cases}$$

Tips: Beräkna tidsintervallen i minuter mellan stationerna, med hänsyn tagen till av- och påstigningstiderna (se tidigare givna modellen) och lägg upp dem i en lista.

Gör en funktion som beräknar n stycken klockslag bestämda av startklockslaget och tidsintervallen. Lägg upp dem i en lista. Använd strukturen ovan för att beräkna tiden δ som det tar för ett tåg att gå hela linjen fram & tillbaka och vara startklar på nytt. Avgångstiderna för hela dagen från en viss station kan sedan lätt beräknas med hjälp av δ och morgonturens tider.

Lägg upp varje stations avgångstider i en lista. I utskriften ska stationens namn stå längst till vänster med ca 10 klockslag per rad. Tänk på att antalet avgångar kan vara större än vad som får plats på en rad. Utskriften får i detta fall delas upp på flera sidor. Utskriftsformatet för tiderna: 07.52, 08.04; tänk på nollorna!

Var noga med att kolla på B-uppgiften innan du designar dina klasser, det är viktigt att du har en struktur som passar med de högre kraven.

Extrauppgift, betyg C: Låt användaren kunna ange nya värden för tåget än de som finns på fil. Inför felkontroll av indata.

Extrauppgift, betyg B: För att göra uppgiften intressantare så ska du nu skilja på vardagar och helgdagar, den tidigare beskrivningen gäller nu för helgdagar och följande gäller för vardagar.

Vardagar

Nu finns det tre tåg i trafik (tre ggr tätare trafik). Första avgång från Arby är kl 05.07. De övriga avgångarna ska ligga så att jämna intervall mellan tågen erhålls. Alla tåg utgår från Arby, dvs morgontåget från Kåvik kan inte avgå förrän 10 minuter efter att det första tåget från Arby kommit dit. I övrigt gäller samma kriterier som för helgdagarna.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till programmet!

140 Sålda biobiljetter

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, datastrukturer, sortering.

Du har fått i uppgift att hjälpa din rika vän att hålla koll på hens alla biografer.

Med hjälp av ditt program ska användaren kunna få en lista med info om alla biografer presenterad, lägga till och ta bort biografer (när de köps och säljs), samt mata in biljettförsäljningen för kvällen innan för en eller alla biografer och sedan även visa statistik över biljettförsäljningen.

Skriv först ihop en fil med följande information för varje biograf:

biografens namn

Totalt antal platser i salongen

Biljettpriser

Man har tre olika avgiftsklasser: vuxna, pensionärer och barn. Biljettpriserna varierar från biograf till biograf. Vuxenbiljetten är alltid dyrast och barnbiljetten billigast och priserna är alltid i hela kronor.

Ditt program ska läsa in informationen om biograferna från filen. Sedan ska användaren få följande val:

- 1 Visa info om alla biografer
- 2 Mata in biljettförsäljningssiffror för alla biografer
- 3 Mata in biljettförsäljningssiffror för en vald biograf
- 4 Visa försäljningsstatistik
- 5 Lägg till en biograf
- 6 Ta bort en biograf
- 7 Spara och avsluta

Infon om biograferna ska på ett lättläst sätt presentera den datan som finns på fil för användaren. När användaren väljer att mata in biljettförsäljningssiffror för alla biografer ska användaren frågas om antal sålda biljetter i varje prisklass, för en biograf i taget. Motsvarande gäller för val 3, men då ska användaren få välja en specifik biograf att mata in biljettförsäljningssiffror för.

Försäljningsstatistiken ska, för varje biograf, bestå av summan av biljettintäkterna och beläggning (i procent). Biograferna ska sorteras och presenteras i fallande ordning efter beläggning (den som har utsålt hamnar alltså först). Dessutom ska summan av alla biografers biljettintäkter skrivas ut.

Såklart ska eventuella uppdateringar i vilka biografer som finns i användarens ägor sparas på fil mellan programkörningarna.

Extrauppgift, betyg C: Inför felkontroll av användarens inmatning. Kontrollera också att filen existerar.

Extrauppgift, betyg B: Biljettförsäljaren räknar ihop kassan efter föreställningen och vill ur kassans storlek avgöra hur många biljetter av varje sort som har sålts. Det kan finnas många lösningar till detta problem och ibland ingen alls (på grund av felräkning i kassan).

Givet biljettpriserna samt kassans storlek ska programmet beräkna alla möjliga lösningar.

För exemplet ovan med tre prisklasser skulle vi få problemet:

Hitta alla heltalslösningar som uppfyller

$$antvuxna \cdot vuxpris + antpens \cdot penspris + antibarn \cdot barnpris = kassan$$

Men i denna deluppgift kan *antalet prisklasser* variera. Någon biograf kan ha en enda prisklass, en annan kanske har fyra eller ännu fler.

Tips: Testa med små exempel först! Tar det lång tid att göra beräkningarna? Fundera över vilka permutationer som är rimliga att testa.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till programmet.

141 Tennismatch

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, sortering, viss filhantering.

Du ska skriva ett program för att samla statistik från tennismatcher. Användaren får välja två spelare som ska spela en match ur en spelarförteckning. Skapa en textfil med spelarnas data. Denna textfil skriver du ihop själv. Den kan se ut så här:

```
Format:
Namn (max 20 tkn) / sannolikhet att vinna sin serve (0-1)
antal vunna matcher / antal spelade matcher
=====
S Halep
0.85
5
7
J Larsson
0.12
:
```

Programmet presenterar urvalet (se nedan) och användaren väljer ut två av spelarna, som får möta varandra i tennismatchen. Användaren får också mata in vem som vann matchen. Sen ska resultatlistan uppdateras och presenteras på nytt. Spelarna ska sorteras med avseende på vinstprocenten. Det kan t.ex. se ut så här:

Placering	Namn	vunna	spelade	andel vunna
1	S Halep	5	7	0.714
2	S Peng	12	34	0.353
3	J Larsson	0	12	0.000
:				

Extrauppgift, betyg C: Kontrollera att infilen existerar och att den innehåller rimliga data. Felhantera även all inmatning som användaren gör så att programmet inte går att krascha.

Extrauppgift, betyg B: Skriv ett program som simulerar en tennismatch i tre set. Matchen delas upp i set, game och bollar. För att avgöra vem som vinner en boll används random. Servar spelare A och $0 < x < p_A$ så vinner A bollen (p_A = sannolikheten att A vinner sin serve). Efter första vinstbollen har man 15 poäng, efter andra 30 och efter den tredje 40. Vidare finns begreppen "lika" och "fördel" samt "game".

Vi illustrerar dessa enklast med ett par exempel på poängställningar, (vi kallar spelarna här för "spelare A" respektive "spelare B"):

```
15 - 0, 15 lika, 15 - 30, 30 lika, 30 - 40
game: spelare B
```

Efter ställningen 40 lika, används endast begreppen "lika", "fördel" och "game" för att beskriva ställningen (det finns alltså inga poängsiffror > 40). "fördel spelare A" betyder att spelare A leder med en boll.

```
30 - 40, 40 lika, fördel spelare A
game: spelare A
```

Gamet är avslutat först efter att en spelare har vunnit, dvs fått 40 poäng och vinner nästföljande boll och motståndaren har < 40 poäng, eller att någon har haft fördel och vinner nästföljande boll. Spelarna serverar vartannat game. Den spelare som först har vunnit 6 game och har minst två game mer än motståndaren (6-4, 6-3 etc), vinner setet. Setet fortsätter tills någon av spelarna har två games övervikt. Matchen spelas i tre set. Vem som börjar att serva avgörs av slumpen.

Användaren skall kunna välja hur ofta hen vill se ställningen, efter varje boll, varje game eller set. Poängställningen skall presenteras på ett trevligt sätt. Efter avslutad match ska setsiffror samt vinnaren presenteras.

Skriv en pausfunktion, så att vi hinner med att se hela presentationen (t.ex. en fördröjning efter varje boll eller game, ett tips är att titta på Pythonmodulen `time`). När användaren ska peka ut en spelare (vilka som ska spela), är det praktiskt att identifiera dem med placeringsnumret. Då slipper användaren mata in hela namnet ...

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt (GUI).

När tennismatcher visas på TV brukar en resultattabell, som ändras efter varje boll och set, visas längst ner i bild. Låt ditt program skriva ut och uppdatera en sådan tabell i ett grafikfönster (istället för att skriva ut resultaten rad för rad).

145 Varuprisdatabas

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering samt datastrukturer.

I många butiker är varorna inte prismärkta utan bär i stället ett streckkodat varunummer som kassaexpediten läser av med streckkodsläsare. I butiksdatorns databas finns varans data lagrade på en textfil som kan se ut så här:

```
% Format:
% kod
% namn
% pris;antal
=====
100
CHIPS
14.90;353
135
STÖVLAR
159;234
:
```

Expediten läser av varorna med sin streckkodsläsare, för att få ett kvitto avslutar denne med att mata in ett #-tecken. Så här kan kvittot se ut:

Varunamn	Antal	A-pris	Summa
CHIPS	1	14.90	14.90
VOLVO	2	502000.00	1004000.00
STÖVLAR	1	159.90	159.90
CHIPS	1	14.90	14.90
Total	5		1004190.00

Ditt program ska uppföra sig på motsvarande sätt. Databasen ska ligga i en varufil, som du skriver ihop själv. Då vi saknar streckkodsläsare matar vi in varorna via kassaapparatens tangentbord. Inmatningen av det ovanstående inköpet får då följande utseende:

```
100
280 2
135
100
#
```

För att hålla antalet filläsningar nere ska du läsa in varuflen i en datastruktur. Detta betyder att du uppdaterar datastrukturen kontinuerligt och varuflen endast efter avslutad körning.

Extrauppgift, betyg C: Tänk på att om det finns 20 påsar chips i lager, ska det inte vara OK att mata in först 14 påsar, och sedan 10 till!

Allt eftersom man slår in varunummer kollar programmet att koden finns i databasen samt att det finns tillräckligt antal varor i lager, annars kommer en felutskrift, följt av möjligheten att mata in på nytt.

Kontrollera också att inmatningens syntax är korrekt så att inga tokigheter händer om man råkar tryck på fel knapp.

Extrauppgift, betyg B: Se till att alla varor av samma slag hamnar på samma plats på kvittot! Det innebär att ovanstående inköp skulle ge följande kvitto i stället:

Varunamn	Antal	A-pris	Summa
CHIPS	2	14.90	14.90
VOLVO	2	502000.00	1004000.00
STÖVLAR	1	159.90	159.90
Total	5		1004190.00

Det händer att kassaexpediten gör felslag. Modifiera ditt program så att det går att ångra inmatade inköp innan kvittot skrivs ut. Man ska givetvis inte (som i riktiga butiker) behöva ångra allt man matat in efter det felaktiga först. Det ska också vara möjligt att ångra bara en del av sitt inköp, t ex ångra 3 påsar chips, när man köpt 5.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt genom vilket all in- och utmatning sker.

154 Wumpus

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, slumpstal, filhantering.

Wumpus är ett enkelt äventyrsspel. Så här ser det ut när man spelar:

Du befinner dig i kulvertarna under D-huset, där den glupske Wumpus bor. För att undvika att bli uppäten måste du skjuta Wumpus med din pil och båge. Kulvertarna har ett antal rum som är förenade med smala gångar. Du kan röra dig åt norr, öster, söder eller väster från ett rum till ett annat.

Här finns dock faror som lurar. I vissa rum finns bottenlösa hål. Kliver du ner i ett sådant dör du omedelbart. I andra rum finns fladdermöss som lyfter upp dig, flyger en bit och släpper dig i ett godtyckligt rum. I ett av rummen finns Wumpus, och om du vågar dig in i det rummet blir du genast uppäten. Som tur är kan du från rummen bredvid känna vinddraget från ett avgrundshål eller lukten av Wumpus. Du får också i varje rum reda på vilka rum som ligger intill.

För att vinna spelet måste du skjuta Wumpus. När du skjuter iväg en pil förflyttar den sig genom tre rum - du kan styra vilken riktning pilen ska välja i varje rum. Glöm inte bort att tunnlarna vindlar sig på oväntade sätt. Du kan råka skjuta dig själv...

Du har fem pilar. Lycka till!

Du hör fladdermöss!

Härifrån kan man komma till följande rum: 6 3 2 14

Vill du förflytta dig eller skjuta (F/S)? *F*

Vilken riktning (N, S, V, Ö)? *N*

Du känner fladdermusvingar mot kinden och lyfts uppåt

Efter en kort flygtur släpper fladdermössen ner dig i rum 19

Du känner lukten av Wumpus!

Du känner vinddrag!

Härifrån kan man komma till följande rum: 2 18 5 15

Vill du förflytta dig eller skjuta (F/S)? *S*

Pilen lämnar första rummet. Vilken riktning (N, S, V, Ö)? *N*

Pilen lämnar andra rummet. Vilken riktning (N, S, V, Ö)? *Ö*

Pilen lämnar tredje rummet. Vilken riktning (N, S, V, Ö)? *N*

Vill du förflytta dig eller skjuta (F/S)? *F*

Vilken riktning? *S*

Du klev just ner i ett bottenlöst hål.

Spelaren (den som kör programmet) går runt i kulvertarna, letar rätt på och försöker skjuta Wumpus. Spelet avslutas när spelaren eller Wumpus dör.

I programmets början ska rummens innehåll slumpas fram. Ungefär 20 % av rummen ska innehålla avgrundshål, 30 % ska vara bebodda av fladdermöss och i ett av rummen finns Wumpus. Ett rum kan aldrig innehålla flera faror (Wumpus äter fladdermöss och varken Wumpus eller fladdermössen gillar draget från avgrundshål).

Kulvertarna ska sättas samman i början av programmet. Du kan använda de givna csv-filerna som förslag på hur rummen ska sitta ihop.

Filen *sgrubben.csv* har tre rum. Första raden motsvarar rum 0, andra raden rum 1 och tredje raden rum 2. Kolumnerna motsvarar i tur och ordning väderstrecken NÖSV. Om vi tittar på andra raden i exemplet nedan kan vi se att om vi går åt Norr kommer vi till rum 0. Går vi åt Öster kommer vi till rum 2. Går vi åt Söder eller Väster kommer vi tillbaka till rum 1.

```
0;0;1;0
0;2;1;1
2;2;2;1
```

Extrauppgift, betyg C: Inför felkontroll för all inmatning från användaren.

Extrauppgift, betyg B: Låt användaren välja svårighetsgrad. Spelet kan göras enklare genom att man sänker sannolikheten för faror, och knepigare genom att låta Wumpus röra sig i kulvertarna. Läskigast blir det om Wumpus hela tiden kommer närmare spelaren, se till att det blir så för högsta svårighetsgraden.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt (GUI). Du behöver inte slumpa rummen som i grunduppgiften.

Datafiler och hjälpfiler: Wumpus

155 Tittarsiffror

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, sökning, sortering, filhantering.

Många är intresserade av TV:s tittarsiffror, däribland programmakare, programchefer och sponsorer. Du ska skriva ett program som presenterar statistik över tittarsiffror på olika sätt. Så här ska det se ut:

```
----- Meny -----
  1. Tio-i-topp-lista
  2. Tittarsiffror för ett visst program
  3. Sluta
  Vad vill du göra? 1

----- Tio-i-topp-lista -----
  1. Aktuellt 57 st (60%)
  2. Sportnytt 47 st (49%)
  3. Rapport med väder 34 st (36%)
  4. Ängeln och den laglöse 33 st (35%)
  5. Tippen 20 st (21%)
  6. Myggan 18 st (19%)
  7. Skilda världar 18 st (19%)
  8. Grannar 16 st (17%)
  9. Paradise Beach 16 st (17%)
 10. Norrköpings sommarcafe 12 st (13%)
  Statistik har samlats in från 93 TV-apparater.
-----
  Vad vill du göra? 2
```

Ditt underlag får du ur följande två filer: Filen `tittardata.txt` som innehåller data för en dag insamlade från de TV-apparater som är med i undersökningen. När TV:n är påslagen registrerar den klockslag och inställd kanal vid fasta tidpunkter varje dag. Data från olika apparater ligger i samma fil, men skiljs åt av streckade rader.

Format: tid/kanal

=====

19.37/2

19.52/2

21.07/1

16.22/4

16.37/4

Filen `program.txt` innehåller en dags TV-program för flera kanaler (åtskilda av streckade rader). Du får utgå från att filen inte innehåller samma program mer än en gång, samt att inget program går över dygnsgränsen. Det är också fritt fram att ändra innehållet bäst man vill bara filen är minst lika stor.

Format: kanal tid program

=====

Kanal 1

21.30-21.40 Sportnytt

21.40-22.35 UR: Sommarkvällar med Tidernas Europa

Kanal 2

8.30-9.00 Let's Dance

9.00-10.00 Biggest Loser

Datafiler och hjälpfiler: tittardata.txt

program.txt

Extrauppgift, betyg C: Inför felhantering för användarens inmatning.

Extrauppgift, betyg B: Man ska nu också kunna se stapeldiagram över viss kanal

Vilken kanal vill du se stapeldiagram för? 2

----- Stapeldiagram över antal tittare, kanal 2 -----

8.30- 9.00 Let's Dance	*****
9.00-10.00 Biggest Loser	*****
17.50-18.20 Klockan Nio: hos stjärnorna	*
18.20-19.20 Friends	*
19.20-19.30 Regionala nyheter	**
19.30-20.00 Rapport med väder	*****
20.00-21.00 Fresh Prince of Bel-Air	*****
21.00-22.25 Teenage mutant ninja turtles	**
22.25-23.15 Hawaii five-0	**
23.15-23.55 Kalla fakta	**

(En stjärna motsvarar 0.9 tittare och längsta stapeln 34 tittare.)

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till programmet. Använd knappar för att ange olika val och presentera statistiken i snygga diagram.

170 Aritmatrisen

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, slumpstal.

En trivsam lekplats för Ture Teknolog och hans glada vänner, som givetvis vill använda sina raster på KTH till att ytterligare förbättra sina matematiska förmågor, är den så kallade Aritmatrisen. Aritmatrisen är en kvadratisk matris med ca 4 rutors sida, som lekledaren Osquar ritat upp med krita på marken. I var och en av matrisens rutor har han skrivit ett någorlunda stokastiskt tal i intervallet 1 - 100. När nu Ture kommer och vill leka, hittar Osquar genast på ett till stokastiskt tal, detta i intervallet 1 - 300, som han säger till Ture. Nu är det Tures uppgift att, påhejad av entusiastiska kamrater, hoppa mellan rutorna i matrisen och med hjälp av +, - och talen han hoppar till försöka bilda det tal Osquar sa.

Du ska skriva ett program som simulerar den här leken på så sätt att användaren får se Aritmatrisen med alla dess siffror på skärmen. Ture står på matrisen, markerad på något lämpligt sätt. OBS! Det är inte lämpligt att täcka siffran med Ture. Båda ska synas samtidigt. Användarens roll blir att skriva instruktioner för hur Ture ska hoppa, och om han ska använda + eller -. En körning kan se ut så här:

15	98	75	82
22	19	8	83
73	91	*35*	99
64	50	37	65

u=upp, n=ner, h=höger, v=vänster u2=upp uv=upp-till-vänster osv

Du kan hoppa högst 2 rutor åt gången.

Avsluta genom att hoppa ut ur matrisen!

Om du ledsnar och vill avbryta, ge a!

Du ska försöka få 231. Just nu har du 35.

Vart vill du hoppa? *uh*

+ eller -? +

Här har spelet just börjat, med att Tures startposition slumpats ut i matrisen. * visar Tures plats. Nästa gång matrisen visas, kommer rutan där 83 står att vara markerad, och texten kommer att visa att användaren samlat på sig 118 (= 35 + 83) poäng. Spelet avbryts genom att användaren låter Ture hoppa ur matrisen, eller ger något avbrottstecken (a ovan). Om han hoppar ut och då har lyckats få det givna antalet poäng (här 231), har han klarat uppgiften, och belönas med att få veta det, samt hur många hopp han behövde.

På grund av begränsad benlängd, kan Ture inte hoppa längre än två rutor horisontellt eller vertikalt, eller en ruta diagonalt. Man ska inte kunna hoppa på stället, dvs räkna en ruta två gånger i följd.

Du ska inte lösa uppgiften genom att låta användaren ange koordinater för den ruta han vill hoppa till, utan enligt exemplet genom antal steg åt ett visst håll utgående från den position han för tillfället står på.

Matrisens storlek ska givetvis vara lätt modifierbar.

Extrauppgift, betyg C: Se till att användarens inmatning kontrolleras.

Extrauppgift, betyg B: Se till att problemet alltid är lösbart (d v s att talet Osquar säger alltid kan fås)! Detta kan göras på flera sätt, men ganska enkelt är att utgå från att om man kan få talet 1 på något vis, går problemet att lösa. I så fall är det ju nämligen bara att hoppa till sig en mängd ettor tills man fått den önskade summan.

Hur ser man då till att det alltid går att få 1? Så enkelt som att alltid slumpa fram minst en etta på planen får du inte göra det för dig. Nej, då blir lösningen för lättgenomskådad. Gör i stället så här:

- Skapa planen som förut, med ett tal på varje plats i matrisen.
- Välj slumpvis en ruta i matrisen. Vi kallar denna ruta A.
- Börja på en annan ruta än A och simulera några (ca 5) stokastiska hopp inom matrisen med slumpvis plus och minus. Se till att inte hoppa på A! Nu har du fått summan S.
- Om $S = 1$, ändra ingenting.
- Om $S \neq 1$ men S ligger i samma intervall som talen på planen (dvs 1 - 100), sätt talet i A till $S - 1$!
- Om S ligger utanför intervallet, fortsätt att hoppa stokastiskt, men nu med plus om $S < 1$ och minus om $S > 100$, tills du hamnar inom intervallet. Gör sedan enligt föregående punkter!

Fundera igenom algoritmen. Du ska ha förstått hur den fungerar vid redovisningen!

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till ditt program så att användaren kan klicka med musen på den ruta han vill hoppa till.

166 Telefonregister

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, sökning, sortering, filhantering.

Du ska skriva ett program som hanterar ett register där man snabbt kan leta upp mobilnumret eller epostadressen för en viss person. Programmet ska också kunna ta reda på vem som har ett visst mobilnummer eller en viss epostadress. Man vill också kunna lägga till personer i registret, ta bort personer ur registret samt kunna ändra en persons mobilnummer, epost och adress. Ditt program skall:

1. Läs in ett register från en fil med namn, mobilnummer, epost och adress för ett antal personer.
2. Så länge användaren vill köra så skall programmet fråga om man vill leta efter ett mobilnummer, en epostadress eller ett namn, lägga till nya uppgifter, ta bort uppgifter, ändra uppgifter (mobilnummer, epost och adress) eller ha en sorterad (namnordning) utskrift av registret.
3. Innan programmet slutar skriva ut uppgifterna på fil igen, inklusive eventuella nya uppgifter och ändringar.

Filen med personuppgifterna skriver du själv in. Du bör ha minst 15 olika personer. Använd en textfil men en rad för varje person. De olika raderna innehåller efternamn, förnamn, mobilnummer, epost och adress, separerade med semikolon.

Exempel:

```
Semla;Simon;079-87654321;simsem@kth.se;Testvägen 4, Huddinge
Mbeki;Lesedi;070-12345678;lesedi23@gmail.com;Klutvägen 3, Bromma
...
```

Den sorterade listan skrivs ut i en fin formaterad tabell, lämpligen med en person på varje rad:

Efternamn	Förnamn	Mobil	Epost	Adress
Mbeki	Lesedi	070-12345678	lesedi23@gmail.com	Klutvägen 3, Bromma
Semla	Simon	079-87654321	simsem@kth.se	Testvägen 4, Huddinge
...				

När man ändrar data om en person, vill man inte vara tvungen att skriva in det som är oförändrat (t ex adressen, om man bara ska byta mobilnummer) på nytt, utan bara det som ska ändras. Det är viktigt för sökningen att uppgifter som skrivs in av användaren, både vid nyinmatning, ändring och sökning, omvandlas till ett enhetligt format. Se till att ta bort inledande och avslutande blankslag, och se till att alla mobilnummer får samma form.

Sökning efter ett visst mobilnummer går till så att användaren matar in ett mobilnummer, programmet letar upp personen med det mobilnumret och sedan skrivs namn, adress och mobilnummer ut för personen. Sökning efter epost och namn går till på motsvarande sätt.

Ändring av en persons uppgifter går till så att man matar in personens för och efternamn. Motsvarande person söks upp och programmet läser in de nya uppgifterna.

När man kör programmet skall man kunna få se en hjälptext genom att ange något kommando, t.ex. ett '??'.

Tips: Skriv programmet i etapper. Börja med att läsa in personerna från fil och skriva ut dem igen. Utöka sedan programmet stegvis.

Extrauppgift, betyg C: Inför felkontroll för användarens inmatning och filers existens.

Extrauppgift, betyg B: Man kan ju vilja ha flera register, t ex ett över badmintonklubben man är ordförande i och ett privat, över vänner. Se till att man kan ha flera register med olika titlar. I början av körningen ska man få välja:

- Använda ett register (som i grunduppgiften). Man kommer vidare till en meny med registernamn. I denna ska man på ett enkelt sätt (inte genom att skriva hela registernamnet) kunna välja ett register eller att gå tillbaka till huvudmenyn. De olika registren lagras i olika filer, lämpligen med namn som 'badminton.reg' och 'vanner.reg' om man har registren 'badminton' och 'vänner'. Byt ut å,ä och ö i filnamnen. Du ska också ha en fil med register över registren, d v s med alla registernamnen. Denna fil läser du in när programmet startar. Om användaren sedan väljer att använda ett register, laddas det valda in.
- Skapa ett nytt register. Se till att ett register med det valda namnet inte finns redan.
- Samköra två register. Här ska menyn med registernamn (se ovan) komma upp. Man ska få välja två register, och sedan komma vidare till menyn:
Lista på alla som finns i båda (de valda) registren.
Lista på alla som finns i något av (de valda) registren, utan dubletter (union).
Tillbaka till registernamnsmenyn. Du får utgå från att om en person finns med i flera register, så har han samma personuppgifter i alla.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till ditt program.

172 Hungriga huggormar

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, slumpstal, grafik

Hilda och Hilding Huggorm är två små, nykläckta ormyngel. Platsen dom lever på är den lilla fyrkantiga Ormön där dom börjar äta grodlår som ormar brukar göra. En groda ökar deras längd med en längdenhet. Födottillgången är varierande ; som mest kan H. och H. sätta i sig 3 grodor åt gången. Ibland kanske de bara får tag på en groda.

För att inte tära för mycket på grodtillgången (dåliga grodår = inga grodlår) får inte Hilda och Hilding komma för nära varandra: Gör dom det så slutar dom att växa! Det är naturligtvis inte bra om dom slår knut på sig själv i sitt sökande efter föda. H. och H. är därför noga med att inte korsa sig själva när dom växer och växer och växer...

Ön vid spelets början ser kanske ut så här (i ett separat grafikfönster):

* = Här ligger Hilding nykläckt och hungrig.
 + = Här ligger Hilda med lika glupande aptit.
 tom ruta = Ingen orm här.

	1	2	3	4	5	6	7	8
A								
B								
C						*		
D								
E								
F								
G			+					
H								

Efter att ha ätit två portioner var kan de ha vuxit ut så här:

- Hilda äter 3 grodor och Hilding 2.
- Hilda äter 1 groda och Hilding 3.

* = Hilding som först vuxit norrut och sedan västerut.
 + = Hilda som vuxit österut och sedan norrut.
 tom ruta = Ingen orm här.

	1	2	3	4	5	6	7	8
A			*	*	*	*		
B						*		
C						*		
D								
E								
F						+		
G			+	+	+	+		
H								

Din uppgift är att skriva ett program som låter två spelare anta Hildas och Hildings roller. Programmet slumpar fram en portion grodor, säg 1 - 3 st, som Hilda äter upp. Programmet frågar Hilda åt vilket håll hon vill växa. Hilda svarar då ett av några möjliga alternativ (hur många då?) Om hon då slår knut på sig själv är spelet över. Likaså om hon inte kan växa åt något håll (om hon då skulle växa ut i vattnet eller korsa Hilding).

Därefter serveras Hilding en portion och programmet upprepar allt som står ovan för Hilding.

Olika personer kan vilja spela spelet med olika inställningar, t.ex. hur många spelare som ska köra samtidigt, namnet på ormarna/spelarna och hur stor planen ska vara så ska du spara dessa inställningar på fil. Antalet spelare kommer i C delen.

Det här skall ditt program göra:

- 1 Slumpa fram 1-3 grodlår till Hilda.
- 2 Hilda (spelare 1) väljer sedan i vilken riktning hon väljer att växa.
- 3 Upprepa 1) - 2) ovan för Hilding (spelare 2).
- 4 Om någon av ormarna inte kan växa vidare är spelet slut. Programmet skall då tala om hur långa Hilda och Hilding blivit. Den orm som förorsakade avbrottet förlorar (storleken har som bekant inte någon betydelse). Om ingen av ormarna kan växa vidare avbryts också spelet och den som är längst vinner.

Extrauppgift, betyg C: Se till att användarens inmatning kontrolleras. Lägg också till att man kan köra flera än två spelare, antalet ska anges i filen med inställningarna och alla spelare ska ha ett eget namn.

Extrauppgift, betyg B: Utöka programmet så att användare 1 (Hilda) kan välja om hon skall spela mot datorn eller mot en annan användare. Låt Hilding (om datorn spelar Hilding) ha en strategi för hur han växer bäst. Denna bör ge intelligent Hilda (som är mån om att kroppen skall vara lång, slank och otrasslig) en god chans att vinna.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt, genom vilket all interaktion sker, så att användaren kan klicka i den riktning ormen ska växa.

174 Patiensen Klockan

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer.

Patiensen Klockan går till så här: Man blandar en vanlig kortlek med 52 kort. Sedan lägger man ut de tretton översta korten i leken i en rad på bordet. Alla kort läggs med framsidan upp, så att man kan se deras valörer.

Nu gäller det att få ett ess överst i den första högen, en tvåa överst i den andra o s v. I elfte högen ska en knekt ligga, i tolfte en dam och i trettonde en kung. Till sin hjälp har man resten av kortleken. Man fortsätter att lägga kort i samma ordning som förut med skillnaden att om rätt valör redan ligger på en plats hoppas denna över. De nya korten täcker delvis de gamla. Patiensen går ut om man lyckas få rätt valör överallt, och misslyckas om högen tar slut utan att patiensen gått ut.

Du ska göra ett program som blandar en kortlek och lägger Klockan. Visa en rad på skärmen för varje varv du lägger i Klockan. En omgång (som inte gick ut) kan se ut så här:

KLOCKAN:

K2	R9	S5	K6	KD	RT	K3	KK	HK	HD	R2	S6	RK
SD	H9	RJ	ST	H3	SK	R7	HT	K8	S2	H6	R8	
SA	HA	R3	H7	HJ	H4		S3	R4	K7	K9	RD	
	K5		S4	S8	S7		S9	SJ	H5	H2		
	H8			RA	KA		K4	R5	R6	KT		
	KJ											

Högen tog slut utan att patiensen gick ut. Låt knekt representeras av bokstaven J, som i engelska ''jackets'' och 10 med bokstaven T.

Här ser man att ruter kung hamnade på rätt plats redan i utläggget. Sedan lades inga fler kort på plats tretton. Ruter sju hamnade rätt på andra utläggget osv. Högen tog slut med klöver knekt. Därför är inga fler kort lagda i sjätte raden. Programmet kommenterar resultatet.

Ditt program ska låta användaren välja mellan:

- Se utlägg av en patiens.
- Se statistik på valt antal patienser (dvs användaren anger ett antal, programmet lägger så många patienser utan att visa utlägg på skärmen och berättar sedan hur många som gick ut)
- Sluta (q)

Extrauppgift, betyg C: Inför felhantering. Om användaren matar in ett felaktigt val ska programmet meddela detta och be om en ny inmatning. Om användaren väljer statistikvalet ska programmet kontrollera att det är ett heltal som användaren matar in, om så inte är fallet ska programmet be om ett nytt värde.

Extrauppgift, betyg B: Du har säkert upptäckt att Klockan inte går ut speciellt ofta. Modifiera ditt program så att när ett kort hamnar på rätt plats på bordet, hamnar de felaktiga kort som låg där innan sist i högen av olagda kort. Dvs när Ruter sju läggs i exemplet ovan, hamnar klöver tre sist i högen av olagda kort. När spader ess läggs, hamnar klöver två näst sist och spader dam sist i högen. (OBS ordningen!) De kort som hamnar sist i högen läggs på så vis ut en gång till på bordet. Egentligen är detta den riktiga versionen av Klockan. Jämför statistikresultaten på den här versionen med resultaten på den gamla!

Extrauppgift, betyg A: Gör ett grafiskt gränssnitt som visar hur klockan ser ut när man lägger den.

När man lägger Klockan med riktiga kort, brukar man i allmänhet lägga korten i form av en urtavla (därav namnet). Esset hamnar då förstas på ettans plats, tvåan på tvåans och så vidare runt till damen på tolvans plats. Kungen får ligga mitt i urtavlan. Inför grafik i ditt program så att utskriften av patiensen blir i form av klockor i stället. Man vill ju fortfarande kunna se alla steg i patiensen, så en klocka per varv måste visas. Ej nylagda kort i varje varv ska vara med, men man ska tydligt se vilka som är nylagda och vilka som inte är det. Exemplet ovan kan då se ut så här:

KLOCKAN:

S6			R8			RD		
R2		K2	H6		SD	K9		SA
HD		R9	S2		H9	K7		HA
HK	RK	S5	K8	(RK)	RJ	R4	(RK)	R3
KK		K6	HT		ST	S3		H7
K3		KD	R7		H3	(R7)		HJ
	RT			SK			H4	
(RD)			(RD)			(RD)		
H2		(SA)	KT		(SA)	(KT)		(SA)
H5		K5	R6		H8	(R6)		KJ
SJ	(RK)	(R3)	R5	(RK)	(R3)	(R5)	(RK)	(R3)
S9		S4	K4		(S4)	(K4)		(S4)
(R7)		S8	(R7)		RA	(R7)		(RA)
	S7			KA			(KA)	

Leken tog slut utan att patiensen gick ut.

178 Periodiska systemet

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, datastrukturer, sortering.

Du ska skriva ett träningsprogram för periodiska systemet. Vid körning kan det till exempel se ut så här.

```
----- MENY -----
1. Visa en lista över alla atomer
2. Träna på atomnummer
3. Träna på atombeteckningar
4. Träna på atomnamn
5. Sluta
-----
Vad vill du göra? 2
```

```
Vilket atomnummer har jod? 35
Fel svar, försök igen.
Vilket atomnummer har jod? 53
Rätt svar!
```

På filen `avikt.txt` finns alla atombeteckningar, atomnamn, atomvikter och atomnummer lagrade. Kopiera den och studera hur den är upplagd. Programmet ska börja med att läsa in atombeteckningar, atomnamn, atomvikter och atomnummer från filen till en datastruktur. Eftersom atomerna i filen är sorterade i bokstavsordning på atombeteckning måste du (för menyval 1) sortera om datastrukturen efter atomnummer för att kunna skriva ut en fin lista.

Programmet ska ge användaren max tre försök på en given fråga, sen ska det rätta svaret visas.

Extrauppgift, betyg C: Lägg till frågor om atomvikten. Ge användaren tre svarsalternativ att välja mellan, varav ett alternativ är rätt och ligger på en slumpmässig plats och de två andra alternativen är felaktiga som slumpas fram varje gång.

Inför även felkontroll för användarens inmatning.

Extrauppgift, betyg B: Det är egentligen viktigare att lära sig i vilken kolumn en atom finns än att lära sig dess atomvikt utantill. Inför rad och kolumn för varje atom och rita upp ett tomt periodiskt system så att rader och kolumner tydligt framgår. Sedan ska programmet be användaren placera in en atom i taget (i slumpvis ordning) på rätt plats tills periodiska systemet är fullständigt.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till programmet!

Datafiler och hjälpfiler: avikt.txt

179 Vilken molekyl?

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, filhantering, datastrukturer, klasser.

Du ska skriva ett program som, givet en procentuell fördelning mellan olika grundämnen, listar ut vilken molekyl det kan vara.

```
Ge ämne och motsvarande andel i procent.
```

```
Ämne och andel: H 11.2
```

```
Ämne och andel: O 88.8
```

```
Ämne och andel:
```

```
Det måste vara H2O.
```

```
Vill du analysera ett nytt ämne? nej
```

På filen `avikt.txt` finns alla atombeteckningar med atomvikter lagrade. Kopiera den och studera hur den är upplagd. Programmet ska börja med att läsa in atombeteckningar och atomvikter från filen till en datastruktur och sedan ta emot användarens inmatning och analysera den.

Den molekyl som programmet kommer fram till ska ha så små koefficienter som möjligt (t ex ska inmatningen $H\ 100$ ge svaret H och inte H_2) och du behöver inte kontrollera om molekylen verkligen existerar. Ta också hänsyn till att procentvärdena är avrundade, så att programmet svarar H_2O och inte $H_{1001}O_{500}$ i exemplet ovan.

- Tips:**
- Dela varje procentsiffra med ämnets atomvikt.
 - Normera alla värden du får då genom att dela med det största (så att alla värden blir mindre eller lika med ett).
 - Om nu alla värden är ungefär ett vet du att det finns lika mycket av varje ämne.
 - Multiplicera annars värdena med två och kontrollera om alla blev ungefär heltal. Om inte, prova att multiplicera med tre osv.

När du provkör programmet kan du skriva ut värdena för att bedöma när de börjar komma nära heltal, men sedan ska programmet avgöra detta.

Exempel att provköra med:

- Propadien (C_3H_4) har 89.9% kol och 10.1% väte.
- Vilken molekyl har fördelningen 37.2% kol, 7.8% väte och 54.9% klor?

Extrauppgift, betyg C: Kontrollera att användarens inmatning är riktig.

Programmet måste kontrollera att bara riktiga grundämneselementbeteckningar matas in och att summan av de inlästa procentvärdena blir ungefär 100.

Extrauppgift, betyg B: Gör ett enkelt stapeldiagram över den procentuella fördelningen mellan de ingående ämnena i terminalfönstret.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt till programmet.

Datafiler och hjälpfiler: `avikt.txt`

184 Patiens med annorlunda kortlek

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer.

Den gamla hederliga kortleken med 52 kort byts i den här uppgiften ut mot en nyskapande kortlek med 81 kort. Varje kort har en färg, en symbol, en fyllnadsgrad och ett visst antal symboler. Om man har tre färger, tre symboler, en till tre olika fyllnadsgrader och antalet symboler är en till tre får man efter lite tänkande fram att det går att skapa 81 olika kort. De 81 olika spelkorterna ska genereras i programmet.

Sex av dessa kort läggs upp på ett spelbord och sedan ska användaren försöka plocka ut tre kort som har något gemensamt, till exempel färgen, eller så ska de tre korten vara helt olika. Om man tar tre kort där något är lika får man två poäng och om man tar tre kort som är helt olika får man tre poäng. Om man väljer ut tre kort som inte uppfyller något av kraven straffas man med en poängs avdrag.

En textversion av spelbordet kan se ut som nedan:

Nr.	Färg	Fylln.	Symbol	Antal
1	blå	1	triangel	1
2	gul	3	triangel	2
3	gul	3	kvadrat	3
4	blå	3	triangel	3
5	gul	1	triangel	1
6	blå	3	kvadrat	1

Användaren ska sedan ange vilka kort som ska väljas ut. Programmet ska undersöka om de utvalda korten uppfyller kriterierna för att få poäng och sedan addera dessa till den tidigare poängsumman. När detta är gjort tas tre nya kort ur leken och placeras på de platser på spelbordet där de tre tidigare korten låg.

Efter varje runda skrivs den utdelade poängen, total poäng och hur många kort som är kvar i leken ut. Dessutom ska användaren tillfrågas om spelet ska fortsätta eller om man är nöjd med den poäng man har.

Spelet avslutas av sig själv när kortleken tagit slut. Då skrivs den erhållna poängen ut.

Lägg värdena för de olika poängen och avdragen man kan få i en fil så att det är lätt att ändra på utan att behöva gå in i koden.

Tips: Gör en ordentlig struktur med klasser och fundera med hjälp av papper och penna på hur korten ska genereras.

Tips: Exempel på färger och symboler: gul, blå, röd, triangel, kvadrat och cirkel

Extrauppgift, betyg C: Gör så att programmet kontrollerar att man har valt ett kort i intervallet 1...6 och att samma kort inte har valts två gånger. Sekvensen 1 3 3 ska alltså innebära att användaren får ett felmeddelande och ombeds göra en ny inmatning.

Extrauppgift, betyg B: Låt datorn lägga patiensen!

Uppfinn en algoritm som givet n kort (där n är en multipel av 3) plockar ut den följd av kort-tripplar som ger mest poäng totalt.

Extrauppgift, betyg A: Gör ett grafiskt gränssnitt där man kan klicka på korten för att välja dem.

192 Kalender

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, stränghantering

Som programledare i det populära TV-programmet “Dejten” har du ett hektiskt medieliv med möten med nya och gamla deltagare, inspelningar och inte minst pressen. Din vardag måste organiseras stenhårt. Gör ett program som fungerar som en filofax. Varje “sida” är en textrad med en minnesanteckning av typen

“2018-03-17: Träffa de nya deltagarna inför avsnitt 137”

eller

“2018-04-01: Träffa Svante Djupsund och berätta att han blivit spolad från serien.”.

Du ska minst ha dessa alternativ i din meny:

1. Bläddra framåt
2. Bläddra bakåt
3. Sätt in ny sida
4. Ta bort sidan
5. Visa alla sidor
6. Avsluta

När man sätter in en ny sida ska den placeras in efter datum. Man vill inte behöva bläddra förbi en massa tomma sidor, så alternativen 1 och 2 ska hoppa direkt till nästa datum där det finns en minnesanteckning.

En annan sak som är viktig är att informationen i filofaxen sparas på fil. Man kan fritt välja om man vill spara allt i samma fil eller om man vill ha en fil per dag.

Extrauppgift, betyg C: Inför felhantering för menyalternativ (vid felaktigt menyval ska programmet fråga om). Se till att det datum användaren anger för en ny sida verkligen är ett giltigt datum.

Extrauppgift, betyg B: Inför klockslag (starttid och sluttid) för aktiviteter. Nu ska det gå att ha flera aktiviteter per dag, och man ska kunna ändra och ta bort enskilda aktiviteter från en viss dag.

Varna om användaren försöker lägga in en aktivitet som överlappar med en annan.

Man ska också kunna se hela månadens aktiviteter.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt där man kan klicka för att bläddra sig fram i filofaxen. Aktuell sida vid start ska vara dagens datum.

Datafiler och hjälpfiler: tidochdatum.txt

193 Memory

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, inläsning från tangentbord och filhantering.

Du ska skriva ett program som spelar Memory. Användaren ska försöka lösa en $A \times A$ (6 x 6 i exemplet) matris genom att matcha ord i olika celler i en matris. Så här kan det se ut:

```
      1   2   3   4   5   6
A --- --- --- --- --- ---
B --- --- --- --- --- ---
C --- --- --- --- --- ---
D --- --- --- --- --- ---
E --- --- --- --- --- ---
F --- --- --- --- --- ---
=====
```

val1: A5

```
      1   2   3   4   5   6
A --- --- --- --- kul ---
B --- --- --- --- --- ---
C --- --- --- --- --- ---
D --- --- --- --- --- ---
E --- --- --- --- --- ---
F --- --- --- --- --- ---
```

val2: F6

```
      1   2   3   4   5   6
A --- --- --- --- kul ---
B --- --- --- --- --- ---
C --- --- --- --- --- ---
D --- --- --- --- --- ---
E --- --- --- --- --- ---
F --- --- --- --- --- ful
```

=====

val1: D1

	1	2	3	4	5	6
A	---	---	---	---	---	---
B	---	---	---	---	---	---
C	---	---	---	---	---	---
D	kul	---	---	---	---	---
E	---	---	---	---	---	---
F	---	---	---	---	---	---

val2: A5

	1	2	3	4	5	6
A	---	---	---	---	kul	---
B	---	---	---	---	---	---
C	---	---	---	---	---	---
D	kul	---	---	---	---	---
E	---	---	---	---	---	---
F	---	---	---	---	---	---

du klarade 1 par.

=====

osv....

Programmet uppmanar användaren att välja två celler. Varje gång användaren väljer en cell skrivs hela matrisen ut på skärmen där ordet i cellen blir synlig (kolla bilden). Sedan kontrollerar programmet om de två valda celler innehåller samma ord och i så fall visas båda orden när matrisen skrivs ut under resten av programmet. Orden göms självklart i fall de inte är samma.

Filen *memo.txt* innehåller 732 ord på var sin rad och varje ord består exakt av 3 bokstäver. Programmet ska slumpa n ord från filen (18 i exemplet ovan), kopiera varje ord och placera ut orden i en matris som sedan används av programmet. Programmet avslutas när användaren har lyckats visa alla ord i matrisen.

Extrauppgift, betyg C: Inför felhantering.

Extrauppgift, betyg B: Gör så att det går att använda ord av varierande längd. Ordlängden ska förstås inte framgå av bilden. Detta kan enklast fixas genom att man sätter antalet streck till längsta ordet.

Inför också en highscore-lista. När en spelare matchat alla ord, ska programmet skriva ut antal försök användaren gjort och visa hens placering på highscore-listan, som också ska sparas på fil mellan programkörningarna. Tänk här även på att storleken på matrisen gör spelet olika svårt, detta bör också sparas på fil och ha betydelse för placeringen.

Extrauppgift, betyg A: Gör en grafisk version av spelet.

Datafiler och hjälpfiler: *memo.txt*

198 Salladsbaren

P-uppgiften ska göras individuellt. Läs EECS:s hederskodex innan du börjar!

Varudeklaration: Klasser, datastrukturer, inläsning från tangentbord och filhantering.

Din kompis "Kalle på hörnet" driver en vegetarisk restaurang som specialiserar sig på sallader. Han har bett dig att skriva ett litet program där folk får välja och vraka bland alla deras olika alternativ.

Programmet ska ha följande steg:

- En person ska ange vad de vill ha i sin sallad.
- Om det finns flera sallader som stämmer överens med de alla de valda ingredienserna så ska alla de sallader som matchandes ges som alternativ till användaren, så får användaren välja vilken hen vill ha. Finns det bara en sallad som har alla de önskade ingredienserna väljs den automatiskt.
- Ifall det inte finns någon sallad som innehåller alla önskade ingredienser så ska den sallad som har flest matchande ingredienser föreslås tillsammans med en lista över vilka ingredienser som behöver kompletteras och totalkostnaden. (Finns det flera sallader som kan komma ifråga ska den billigaste väljas.)
- Efter att personen har valt sin sallad så ska man komma till menyn för extra val där alla ingredienser och deras priser listas.
- När personen lagt till ingredienser (eller valt att inte göra det) och är nöjd så ska ett kvitto skrivas ut på fil.

När programmet startar så ska du läsa in alla sallader från en fil. Det som ska finnas med för varje sallad är ett namn, pris och ingredienser. Utöver detta så får du strukturera filen hur du vill.

Det ska också finnas en separat fil där alla ingredienser står var för sig med ett pris. Denna data laddas in och används sen när man ska lägga till enskilda ingredienser till salladen.

Extrauppgift, betyg C: Inför felhantering.

Extrauppgift, betyg B: Du ska nu göra en huvudmeny där du ska kunna välja bland flera olika salladsbarer. Inget av datan för vilka restauranger som finns ska finnas i programmet utan allt ska finnas på fil.

Extrauppgift, betyg A: Gör ett grafiskt användargränssnitt genom vilket all interaktion sker.

